

Answers to quiz #2 on Algebraic Specification

Christian Rinderknecht

3 June 2005

1 Arrays

We want an algebraic specification of arrays. An array is a list whose size is fixed at the construction time and whose elements are items of the same type. There is a special item which is used as a default element when creating a new array (i.e. a freshly created array contains these default elements). Then the user can change an element to another value by specifying the integer index (i.e. position) in the array and the new value.

Let us call ITEM the specification of some items whose signature is as follows.

- **Defined types** The type of the items is noted t
- **Constructors**
DEFAULT : t
The term DEFAULT is a distinguished item, whose interpretation, i.e. meaning, is left to the user of this specification.

Let us call now ARRAY(ITEM) the specification of the arrays over items of type ITEM. t . The signature is as follows.

- **Defined types** The type of the arrays is noted t .
- **Constructors**
 - EMPTY : t
The term EMPTY represents the array which contains no element.
 - CREATE : $\text{int} \times \text{int} \rightarrow t$
The term CREATE(x, y) denotes an array whose elements are indexed from x to y , if $x < y$, or from y to x otherwise. The type int denotes the set of positive integers. Integers x and y are called *bounds*. If $x < y$ then x is the *lower bound* and y is the *upper bound*.
For example, CREATE(3, 5) is an array whose elements are indexed by integers 3, 4 and 5. It contains three elements. These elements are all equal to ITEM.DEFAULT. Also CREATE(5, 3) is the same

as $\text{CREATE}(3, 5)$. Therefore, in both cases, the lower bound is 3 and the upper bound is 5.

– $\text{SET} : \mathbf{t} \times \mathbf{int} \times \mathbf{ITEM.t} \rightarrow \mathbf{t}$

The term $\text{SET}(a, n, e)$ represents an array equal to array a except that the element at index n is e . If n is out of bounds, i.e. if n is not between the bounds of a , the result of SET is unspecified. Same if a is empty.

• Functions

– $\text{LOWER} : \mathbf{t} \rightarrow \mathbf{int}$

The call $\text{LOWER}(a)$ is the lower bound of array a , i.e. the smallest valid index. If a is empty, the result is unspecified.

– $\text{UPPER} : \mathbf{t} \rightarrow \mathbf{int}$

The call $\text{UPPER}(a)$ is the upper bound of array a , i.e. the greatest valid index. If a is empty, the result is unspecified.

– $\text{GET} : \mathbf{t} \times \mathbf{int} \rightarrow \mathbf{ITEM.t}$

The call $\text{GET}(a, n)$ denotes the element in array a at index n . If n is out of bounds or if a is empty, the result of GET is unspecified.

– $\text{NTH} : \mathbf{t} \times \mathbf{int} \rightarrow \mathbf{ITEM.t}$

The call $\text{NTH}(a, n)$ denotes the n th element in array a . If a is empty or n is out of bounds, the call is unspecified.

– $\text{MEM} : \mathbf{t} \times \mathbf{ITEM.t} \rightarrow \mathbf{bool}$

The call $\text{MEM}(a, e)$ is TRUE if e is an element of array a and FALSE otherwise.

– $\text{INV} : \mathbf{t} \rightarrow \mathbf{t}$

The call $\text{INV}(a)$ denotes the array whose elements are in the reverse order as found in a . For example, let a be the array graphically represented as

e_1	e_2	e_3	e_4
-------	-------	-------	-------

, then $\text{INV}(a)$ is represented as

e_4	e_3	e_2	e_1
-------	-------	-------	-------

.

– $\text{UNDO} : \mathbf{t} \times \mathbf{int} \rightarrow \mathbf{t}$

The call $\text{UNDO}(a, n)$ represents the array a without the last modification (using SET) at index n . If a is empty or has not been modified at index n , the call equals a .

Question. Complete this signature with equations defining the functions NTH , MEM , INV and UNDO (remember you can use LOWER , UPPER or GET if you need).

Answer.

- NTH can be expressed in terms of GET as follows:

$$\text{NTH}(a, n) = \text{GET}(a, \text{LOWER}(a) + n - 1), \quad \text{if } a \neq \text{EMPTY}.$$

- $\text{MEM} : \mathbf{t} \times \text{ITEM.t} \rightarrow \text{bool}$

The first argument of MEM is an array, so we can start by envisaging the three constructors of arrays:

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= ? \\ \text{MEM}(\text{CREATE}(x, y), e) &= ? \\ \text{MEM}(\text{SET}(a, n, e_1), e_2) &= ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a)\end{aligned}$$

The first equation refers to an empty array into which we look for some element, so this element, as well as any element, is not present, for sure, so:

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= \text{FALSE} \\ \text{MEM}(\text{CREATE}(x, y), e) &= ? \\ \text{MEM}(\text{SET}(a, n, e_1), e_2) &= ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a)\end{aligned}$$

As far as the second equation is concerned, we know that an unmodified array contains, by construction, only `ITEM.DEFAULT` elements. So we must consider two cases: $e = \text{DEFAULT}$ or not:

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= \text{FALSE} \\ \text{MEM}(\text{CREATE}(x, y), e) &= ? \quad \text{if } e = \text{DEFAULT} \\ \text{MEM}(\text{CREATE}(x, y), e) &= ? \quad \text{if } e \neq \text{DEFAULT} \\ \text{MEM}(\text{SET}(a, n, e_1), e_2) &= ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a)\end{aligned}$$

Or, simply

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= \text{FALSE} \\ \text{MEM}(\text{CREATE}(x, y), \text{DEFAULT}) &= ? \\ \text{MEM}(\text{CREATE}(x, y), e) &= ? \quad \text{if } e \neq \text{DEFAULT} \\ \text{MEM}(\text{SET}(a, n, e_1), e_2) &= ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a)\end{aligned}$$

Now, the first case is simple:

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= \text{FALSE} \\ \text{MEM}(\text{CREATE}(x, y), \text{DEFAULT}) &= \text{TRUE} \\ \text{MEM}(\text{CREATE}(x, y), e) &= ? \quad \text{if } e \neq \text{DEFAULT} \\ \text{MEM}(\text{SET}(a, n, e_1), e_2) &= ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a)\end{aligned}$$

The second one also:

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= \text{FALSE} \\ \text{MEM}(\text{CREATE}(x, y), \text{DEFAULT}) &= \text{TRUE} \\ \text{MEM}(\text{CREATE}(x, y), e) &= \text{FALSE} \quad \text{if } e \neq \text{DEFAULT} \\ \text{MEM}(\text{SET}(a, n, e_1), e_2) &= ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a)\end{aligned}$$

The last equation refers to the case of an array modified at index n , in which we search for element e_2 . The first idea that should occur is to check whether the new element at index n , namely e_1 , is e_2 or not. Indeed, if they are equal, the result should be TRUE. So

$$\begin{aligned}
& \text{MEM}(\text{EMPTY}, e) = \text{FALSE} \\
& \text{MEM}(\text{CREATE}(x, y), \text{DEFAULT}) = \text{TRUE} \\
& \text{MEM}(\text{CREATE}(x, y), e) = \text{FALSE} \quad \text{if } e \neq \text{DEFAULT} \\
& \text{MEM}(\text{SET}(a, n, e_1), e_2) = ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\
& \quad \text{and } e_1 = e_2 \\
& \text{MEM}(\text{SET}(a, n, e_1), e_2) = ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\
& \quad \text{and } e_1 \neq e_2
\end{aligned}$$

Or, simply

$$\begin{aligned}
& \text{MEM}(\text{EMPTY}, e) = \text{FALSE} \\
& \text{MEM}(\text{CREATE}(x, y), \text{DEFAULT}) = \text{TRUE} \\
& \text{MEM}(\text{CREATE}(x, y), e) = \text{FALSE} \quad \text{if } e \neq \text{DEFAULT} \\
& \text{MEM}(\text{SET}(a, n, e), e) = ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\
& \text{MEM}(\text{SET}(a, n, e_1), e_2) = ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\
& \quad \text{if } e_1 \neq e_2
\end{aligned}$$

Then

$$\begin{aligned}
& \text{MEM}(\text{EMPTY}, e) = \text{FALSE} \\
& \text{MEM}(\text{CREATE}(x, y), \text{DEFAULT}) = \text{TRUE} \\
& \text{MEM}(\text{CREATE}(x, y), e) = \text{FALSE} \quad \text{if } e \neq \text{DEFAULT} \\
& \text{MEM}(\text{SET}(a, n, e), e) = \text{TRUE} \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\
& \text{MEM}(\text{SET}(a, n, e_1), e_2) = ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\
& \quad \text{and } e_1 \neq e_2
\end{aligned}$$

The last case refers to a modified array whose new element is different from which the one we look for. What happen if we simply ignore (i.e. undo) the last modification and search again? This means the right-hand side of the last equation would be $\text{MEM}(a, e_2)$.

This would be a problem if a contains e_2 at the same index n , i.e. contains a subterm like $\text{SET}(b, n, e_2)$. This is possible because our arrays remember all the modifications done since their creation. So, searching a instead of $\text{SET}(a, n, e_1)$ means searching in the previous modifications of the array $\text{SET}(a, n, e_1)$. But if e_2 was present at index

n in a , the value of $\text{MEM}(a, e_2)$ will be **TRUE**, *despite we know that in the given array, there is no e_2 at index n .*

So we must find a way to ignore index n in the subsequent searches. The simplest way is to use an auxiliary function $\overline{\text{MEM}} : \mathbf{t} \times \text{ITEM.t} \times \text{STACK(int).t} \rightarrow \text{bool}$. The difference with MEM is that $\overline{\text{MEM}}$ takes a supplementary argument which is a stack of indexes. These indexes are the already visited indexes, so they must be ignored in previous versions of the array, in order to avoid the problem we mentioned in the previous paragraph.

Therefore the definition of MEM is now done in terms of $\overline{\text{MEM}}$:

$$\text{MEM}(a, e) = \overline{\text{MEM}}(a, e, \text{STACK(int).EMPTY})$$

In order to save some space, let us write

- $n \in s$ instead of $\text{STACK(int).MEM}(s, n)$ (we do not define this membership function on stack here, but it is easy to do);
- $n :: s$ instead of $\text{STACK(int).PUSH}(n, s)$

Now we can rewrite our equations with $\overline{\text{MEM}}$ instead of MEM :

$$\begin{aligned} \overline{\text{MEM}}(\text{EMPTY}, e, \mathbf{s}) &= \text{FALSE} \\ \overline{\text{MEM}}(\text{CREATE}(x, y), \text{DEFAULT}, \mathbf{s}) &= \text{TRUE} \\ \overline{\text{MEM}}(\text{CREATE}(x, y), e, \mathbf{s}) &= \text{FALSE} && \text{if } e \neq \text{DEFAULT} \\ \overline{\text{MEM}}(\text{SET}(a, n, e), e, \mathbf{s}) &= \text{TRUE} && \text{if } n \notin \mathbf{s} \\ &&& \text{and } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\ \overline{\text{MEM}}(\text{SET}(a, n, e_1), e_2, \mathbf{s}) &= ? && \text{if } e_1 \neq e_2 \\ &&& \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \end{aligned}$$

We notice that we add a condition $n \notin s$ which means that the index n is visited here for the first time, so we can take the corresponding contents, e , into account (here the result is **TRUE**).

What if $n \in s$? Actually, this situation encompasses whether $e_1 = e_2$ or not: in both cases, we must ignore the current index n and recursively explore the other indexes. This is achieved by the recursive call $\overline{\text{MEM}}(a, e_2, s)$. Technically, we just removed the **SET** call of the left-

hand side $\overline{\text{MEM}}(\text{SET}(a, n, e_1), e_2, s)$. We must add another equation:

$$\begin{aligned}
& \overline{\text{MEM}}(\text{EMPTY}, e, s) = \text{FALSE} \\
& \overline{\text{MEM}}(\text{CREATE}(x, y), \text{DEFAULT}, s) = \text{TRUE} \\
& \overline{\text{MEM}}(\text{CREATE}(x, y), e, s) = \text{FALSE} \quad \text{if } e \neq \text{DEFAULT} \\
& \overline{\text{MEM}}(\text{SET}(a, n, e), e, s) = \text{TRUE} \quad \text{if } n \notin s \\
& \quad \text{and } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\
& \overline{\text{MEM}}(\text{SET}(a, n, e_1), e_2, s) = \overline{\text{MEM}}(a, e_2, s) \quad \text{if } n \in s \\
& \overline{\text{MEM}}(\text{SET}(a, n, e_1), e_2, s) = ? \quad \text{if } e_1 \neq e_2 \text{ and } n \notin s \\
& \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a)
\end{aligned}$$

Notice that the condition $\text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a)$ has been removed because it is a recursive call, so we just can let the non-recursive calls check the bounds (see the fourth equation). Note also that we had the condition $n \notin s$ to the last equation, because we just added the case $n \in s$ in the fourth equation.

The remaining case is also a recursive call because we know that $e_1 \neq e_2$, i.e. the element at index n is not the one we look for, and it is the first time we visit this index n , so we must record it in the stack s of indexes, so we will not consider it again (in the subsequent recursive calls):

$$\begin{aligned}
& \overline{\text{MEM}}(\text{EMPTY}, e, s) = \text{FALSE} \\
& \overline{\text{MEM}}(\text{CREATE}(x, y), \text{DEFAULT}, s) = \text{TRUE} \\
& \overline{\text{MEM}}(\text{CREATE}(x, y), e, s) = \text{FALSE} \quad \text{if } e \neq \text{DEFAULT} \\
& \overline{\text{MEM}}(\text{SET}(a, n, e), e, s) = \text{TRUE} \quad \text{if } n \notin s \\
& \quad \text{and } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\
& \overline{\text{MEM}}(\text{SET}(a, n, e_1), e_2, s) = \overline{\text{MEM}}(a, e_2, s) \quad \text{if } n \in s \\
& \overline{\text{MEM}}(\text{SET}(a, n, e_1), e_2, s) = \overline{\text{MEM}}(a, e_2, n :: s) \quad \text{if } e_1 \neq e_2 \\
& \quad \text{and } n \notin s
\end{aligned}$$

- $\text{INV} : \mathbf{t} \rightarrow \mathbf{t}$

The sole argument of INV is an array, so we can simply start by enumerating all the array constructors:

$$\begin{aligned}
& \text{INV}(\text{EMPTY}) = ? \\
& \text{INV}(\text{CREATE}(x, y)) = ? \\
& \text{INV}(\text{SET}(a, n, e)) = ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a)
\end{aligned}$$

The first equation corresponds to the inversion of an empty array: it is pretty clear that it must be the empty array too:

$$\begin{aligned} \text{INV}(\text{EMPTY}) &= \text{EMPTY} \\ \text{INV}(\text{CREATE}(x, y)) &= ? \\ \text{INV}(\text{SET}(a, n, e)) &= ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \end{aligned}$$

The second equation refers to the case of an unmodified array to be inverted. Since, by construction, all elements of an unmodified array are `ITEM.DEFAULT`, the inverted array is the same as the original one, just as for the empty array. So

$$\begin{aligned} \text{INV}(\text{EMPTY}) &= \text{EMPTY} \\ \text{INV}(\text{CREATE}(x, y)) &= \text{CREATE}(x, y) \\ \text{INV}(\text{SET}(a, n, e)) &= ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \end{aligned}$$

The last equation refers to the case of an array modified at index n . The inverted array has thus to be modified symmetrically, i.e. it must have the same modification but on the symmetric index (starting from the end of the array instead of the beginning). The symmetric index of n in array a is

$$\text{UPPER}(a) - n + \text{LOWER}(a)$$

(Note that modifying an element does not change the bounds.) So we have to do the modification

$$\text{SET}(?, \text{UPPER}(a) - n + \text{LOWER}(a), e)$$

But what array do we must modify? It cannot be a because we have to invert the other elements in a before applying the last symmetric modification. So it must be $\text{INV}(a)$. As a conclusion, we get

$$\begin{aligned} \text{INV}(\text{EMPTY}) &= \text{EMPTY} \\ \text{INV}(\text{CREATE}(x, y)) &= \text{CREATE}(x, y) \\ \text{INV}(\text{SET}(a, n, e)) &= \text{SET}(\text{INV}(a), \text{UPPER}(a) - n + \text{LOWER}(a), e) \\ &\quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \end{aligned}$$

Actually, we could simplify a little bit these equations by noting that if the array is not modified, then the inverted array is invariant. Formally:

$$\begin{aligned} \text{INV}(a) &= a \quad \text{if } a \neq \text{SET}(b, n, e) \\ \text{INV}(\text{SET}(a, n, e)) &= \text{SET}(\text{INV}(a), \text{UPPER}(a) - n + \text{LOWER}(a), e) \\ &\quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \end{aligned}$$

- $\text{UNDO} : \mathbf{t} \times \mathbf{int} \rightarrow \mathbf{t}$

Since the first argument of UNDO is an array and since there are three array constructors, we start by enumerating these cases:

$$\begin{aligned}\text{UNDO}(\text{EMPTY}, n) &= ? \\ \text{UNDO}(\text{CREATE}(x, y), n) &= ? \\ \text{UNDO}(\text{SET}(a, p, e), n) &= ? \quad \text{if } \text{LOWER}(a) \leq p \leq \text{UPPER}(a)\end{aligned}$$

The text says that if the array is empty or unmodified, the result of undoing the last modification is simply the original array:

$$\begin{aligned}\text{UNDO}(\text{EMPTY}, n) &= \text{EMPTY} \\ \text{UNDO}(\text{CREATE}(x, y), n) &= \text{CREATE}(x, y) \\ \text{UNDO}(\text{SET}(a, p, e), n) &= ? \quad \text{if } \text{LOWER}(a) \leq p \leq \text{UPPER}(a)\end{aligned}$$

The remaining equation refers to the case of an array whose last modification has been made at index p . Since we want to undo the last modification at index n , we must check whether $p = n$ or not:

$$\begin{aligned}\text{UNDO}(\text{EMPTY}, n) &= \text{EMPTY} \\ \text{UNDO}(\text{CREATE}(x, y), n) &= \text{CREATE}(x, y) \\ \text{UNDO}(\text{SET}(a, p, e), n) &= ? \quad \begin{array}{l} \text{if } p = n \\ \text{and } \text{LOWER}(a) \leq p \leq \text{UPPER}(a) \end{array} \\ \text{UNDO}(\text{SET}(a, p, e), n) &= ? \quad \begin{array}{l} \text{if } p \neq n \\ \text{and } \text{LOWER}(a) \leq p \leq \text{UPPER}(a) \end{array}\end{aligned}$$

Or, simply

$$\begin{aligned}\text{UNDO}(\text{EMPTY}, n) &= \text{EMPTY} \\ \text{UNDO}(\text{CREATE}(x, y), n) &= \text{CREATE}(x, y) \\ \text{UNDO}(\text{SET}(a, n, e), n) &= ? \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\ \text{UNDO}(\text{SET}(a, p, e), n) &= ? \quad \begin{array}{l} \text{if } p \neq n \\ \text{and } \text{LOWER}(a) \leq p \leq \text{UPPER}(a) \end{array}\end{aligned}$$

If the last modification was done at the index we want to undo, the result is the array without the call to SET :

$$\begin{aligned}\text{UNDO}(\text{EMPTY}, n) &= \text{EMPTY} \\ \text{UNDO}(\text{CREATE}(x, y), n) &= \text{CREATE}(x, y) \\ \text{UNDO}(\text{SET}(a, n, e), n) &= a \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\ \text{UNDO}(\text{SET}(a, p, e), n) &= ? \quad \begin{array}{l} \text{if } p \neq n \\ \text{and } \text{LOWER}(a) \leq p \leq \text{UPPER}(a) \end{array}\end{aligned}$$

Now, the last equation refers to the case where the last modification was done on a different index than the one we want to undo. Therefore, we must keep this modification and undo on the array *without* this modification:

$$\begin{aligned}
& \text{UNDO}(\text{EMPTY}, n) = \text{EMPTY} \\
& \text{UNDO}(\text{CREATE}(x, y), n) = \text{CREATE}(x, y) \\
& \text{UNDO}(\text{SET}(a, n, e), n) = a \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\
& \text{UNDO}(\text{SET}(a, p, e), n) = \text{SET}(\text{UNDO}(a, n), p, e) \quad \text{if } p \neq n \\
& \quad \text{and } \text{LOWER}(a) \leq p \leq \text{UPPER}(a)
\end{aligned}$$

As a final note, we can simplify a little bit more the equations by grouping the two first ones:

$$\begin{aligned}
& \text{UNDO}(a, n) = a \quad \text{if } a \neq \text{SET}(b, p, e) \\
& \text{UNDO}(\text{SET}(a, n, e), n) = a \quad \text{if } \text{LOWER}(a) \leq n \leq \text{UPPER}(a) \\
& \text{UNDO}(\text{SET}(a, p, e), n) = \text{SET}(\text{UNDO}(a, n), p, e) \quad \text{if } p \neq n \\
& \quad \text{and } \text{LOWER}(a) \leq p \leq \text{UPPER}(a)
\end{aligned}$$